

Adder Assignment Report
Course: LLM4ChipDesign
Instructors: Prof. Ramesh Karri

1. Introduction

This project explores the use of Large Language Models (LLMs) in digital hardware design, specifically for arithmetic circuits. The workflow spans three stages: regenerating RTL from existing designs, verifying correctness through simulation, and improving designs using a synthesis-driven optimization loop.

Two 8-bit adder architectures were selected: **RCA8 (Ripple Carry Adder)** and **CLA8 (Carry Lookahead Adder)**. These represent two fundamentally different approaches to carry computation—sequential versus parallel—and provide a useful basis for evaluating both functional correctness and performance trade-offs.

2. Part 1 — LLM Regeneration from Golden RTL

2.1 Adder Selection

- **Adder 1:** RCA8
- **Adder 2:** CLA8

The goal was to compare a simple, easy-to-understand architecture (RCA) with a more optimized design (CLA) that reduces carry propagation delay through additional logic.

2.2 Natural Language Descriptions

- `part1/nl_descriptions/RCA8_description.md`
- `part1/nl_descriptions/CLA8_description.md`

The golden RTL designs were first translated into structured natural language descriptions. These captured:

- module hierarchy
- signal flow
- datapath behavior
- key logic (especially carry computation)

This step is important because it tests whether an LLM can reconstruct hardware from a textual description alone.

2.3 Regenerated Verilog

- part1/llm_generated/RCA8_llm.v
- part1/llm_generated/CLA8_llm.v

Using the descriptions, the LLM generated synthesizable Verilog. Both designs compiled successfully using Icarus Verilog, which confirmed that the outputs were syntactically valid and structurally complete.

2.4 Manual Verification

A manual comparison between golden and generated designs showed:

- Matching module interfaces and hierarchy
- Correct signal connections and carry logic
- Only minor stylistic differences (e.g., constant formatting like 0 vs 1'b0)

Overall, the regenerated designs preserved the intended architecture and were deemed functionally equivalent.

3. Part 2 — Testbenches and Simulation

3.1 Testbenches with Internal Signal Verification

Two testbenches were developed to go beyond simple input-output checking.

RCA8 testbench:

- Verifies outputs (sum, cout)
- Checks internal carry chain (dut.c[7:1])

CLA8 testbench:

- Verifies outputs
- Checks internal signals (g, p, c)

This allowed validation not just of correctness, but of whether the internal implementation matched the expected architecture.

3.2 Simulation Results

- **RCA8:** PASS = 65,536, FAIL = 0
- **CLA8:** PASS = 65,536, FAIL = 0

All possible 8-bit input combinations were tested exhaustively.

3.3 Analysis

- Output values matched expected arithmetic results
- Internal signals behaved as defined by each architecture
- This confirmed correctness at both functional and structural levels

This step was especially useful in distinguishing between designs that are merely functionally correct and those that correctly implement the intended logic.

4. Part 3 — Yosys-Based Optimization Loop

4.1 Setup

An automated optimization loop was implemented by combining LLM-based code generation with Yosys synthesis feedback.

Key components:

- `run_yosys.py` — synthesis + metric extraction
- `optimize_adder.py` — iterative LLM generation
- Shell scripts (*.sh) — run different optimization modes
- Yosys scripts (*.ys) — synthesis flow

Three optimization strategies were explored:

- Area-focused
- Delay-focused
- Balanced

4.2 Optimization Results

All three modes ran successfully and produced valid synthesis outputs.

- **Baseline RCA8:** 40 cells
- **Best result (delay mode):** 37 cells

The optimization logs show multiple iterations with different generated designs, indicating that the LLM explored variations in structure and logic organization.

Even though the improvement is modest, it demonstrates that the loop is functioning correctly and can refine designs.

4.3 Best Design Verification

- Selected design: part3/runs/mode_delay_rca8/best_adder.v

Functional simulation:

- PASS = 65,536, FAIL = 0

Internal-signal testbench:

- Not applicable

Reason:

The optimized design does not preserve the original ripple-carry structure. As a result, internal signals (such as carry chains) differ from the original RCA implementation.

This is expected; optimization changes the implementation, not the functionality.

Formal equivalence:

- Golden vs LLM designs (Part 1): PASS

This confirms that the optimized design is still functionally correct.

5. Reflection and Lessons Learned

- LLMs can generate valid and structured hardware when given clear constraints and context
- However, they are sensitive to details such as module names and interfaces
- Internal-signal validation is important for verifying architectural correctness
- Optimization requires a tight loop between generation and synthesis tools
- LLM outputs should never be trusted without simulation and formal verification

Overall, LLMs are useful for accelerating development, but not for replacing verification.

6. Conclusion

This project demonstrates that LLMs can assist in generating and refining hardware designs, especially when integrated with tools like Yosys.

While LLMs can explore design alternatives quickly, correctness must always be ensured through:

- simulation
- structural validation
- formal equivalence

The combination of LLMs and traditional EDA tools provides a promising workflow for design-space exploration, but requires careful control and validation at every step.